

Fall 2024

Homework 4

STAT 656: Bayesian Data Analysis

Juanwu Lu*

Gibbs Sampling for Probit Model

Consider N pairs of observations (x_i, y_i) , where $x_i \in \mathbf{R}^d$ and $y_i \in \{0, 1\}$. These are generated as follows:

$$z_i \sim \mathcal{N}(x_i\beta, 1) \quad y_i = \mathbb{1}(z_i > 0), \quad i = 1, \dots, N$$

This is thus the probit model for the Lecture slides. Write X , Y , and Z for the collection of x_i , y_i , and z_i 's. We will place a $\mathcal{N}(0, \Sigma_b)$ prior on $\beta \in \mathbf{R}^d$, and draw samples from the posterior $p(\beta, Z \mid X, Y)$. Observe that as in regression settings, we are really only interested in β , the Z 's only auxiliary variables to allow us to implement a Gibbs sampler. In other words, this is an example of a *data-augmentation algorithm*.

1. (7 points) Derive and write down the conditional distribution $p(Z \mid X, Y, \beta)$.
2. (7 points) Derive and write down the conditional distribution $p(\beta \mid X, Y, Z)$?
3. (1 points) Describe the overall Gibbs sampling algorithm, and how you would calculate the posterior mean and covariance of β .
4. (20 points) Write an R function that implement the VB algorithm for the probit model given in the slides. This should accept as input a dataset (X, Y) and return the approximations to the posterior distribution over β and the z 's. Recall how the algorithm proceeds: start with some arbitrary initial distribution $q(\beta)$ over β , use that to calculate the $q(z_i)$'s, use those to calculate $q(\beta)$, and repeat till these distributions stop changing.

Solution:

1. The conditional distribution $Z \mid X, Y, \beta$ is derived as follows:

$$\begin{aligned} \log p(Z \mid X, Y, \beta) &= \log p(\beta) + \sum_{i=1}^N \log p(y_i \mid z_i) + \log p(z_i \mid x_i) + C_1 \\ &= -\frac{1}{2}\beta^T \Sigma_b^{-1} \beta + \sum_{i=1}^N y_i \log \mathbb{1}(z_i > 0) + (1 - y_i) \log \mathbb{1}(z_i \leq 0) - \frac{1}{2}(z_i - x_i\beta)^2 + C_2 \end{aligned} \tag{1}$$

Therefore, the conditional distribution is given by:

$$p(z_i \mid x_i, y_i, \beta) \propto \begin{cases} \mathbb{1}(z_i > 0) \mathcal{N}(x_i\beta, 1) & \text{if } y_i = 1, \\ \mathbb{1}(z_i \leq 0) \mathcal{N}(x_i\beta, 1) & \text{otherwise.} \end{cases} \tag{2}$$

*College of Engineering, Purdue University, West Lafayette, IN, USA

2. Based on the results from equation (1), the conditional distribution of $\beta \mid X, Y, Z$ is derived by:

$$\begin{aligned}
\log p(\beta \mid X, Y, Z) &= -\frac{1}{2} \sum_{i=1}^d z_i^2 - 2z_i(x_i\beta) + x_i\beta\beta^\top x_i^\top + \beta^\top \Sigma_b^{-1} \beta + C_3 \\
&= -\frac{1}{2} \beta^\top \left(\sum_{i=1}^N x_i^\top x_i + \Sigma_b^{-1} \right) \beta - 2 \sum_{i=1}^N z_i x_i \beta + C_4 \\
&= -\frac{1}{2} \left(\beta - \Sigma_{\beta \mid X, Y, Z} \sum_{i=1}^N z_i x_i^\top \right)^\top \Sigma_{\beta \mid X, Y, Z}^{-1} \left(\beta - \Sigma_{\beta \mid X, Y, Z} \sum_{i=1}^N z_i x_i^\top \right) + C_5, \\
\text{where } \Sigma_{\beta \mid X, Y, Z} &= \left(\sum_{i=1}^N x_i^\top x_i + \Sigma_b^{-1} \right)^{-1}
\end{aligned}$$

Therefore, the conditional distribution is a multivariate normal distribution

$$p(\beta \mid X, Y, Z) \sim \mathcal{N} \left(\left(\sum_{i=1}^N x_i^\top x_i + \Sigma_b^{-1} \right)^{-1} \sum_{i=1}^N z_i x_i^\top, \left(\sum_{i=1}^N x_i^\top x_i + \Sigma_b^{-1} \right)^{-1} \right) \quad (2)$$

3. The Gibbs sampling algorithm is as follows:

- Initialize β and Z .
- Repeat the following steps until convergence or a maximum number of iterations:
 - Sample $Z \mid X, Y, \beta$ from the conditional distribution $p(Z \mid X, Y, \beta)$.
 - Sample $\beta \mid X, Y, Z$ from the conditional distribution $p(\beta \mid X, Y, Z)$. Therefore, with every sampled collection of Z , we can calculate the posterior mean and covariance of β according to the results equation (2):
- Calculate the covariance by $\left(\sum_{i=1}^d x_i^\top x_i + \Sigma_b^{-1} \right)^{-1}$.
- Calculate the mean by $\left(\sum_{i=1}^d x_i^\top x_i + \Sigma_b^{-1} \right)^{-1} \sum_{i=1}^d z_i x_i^\top$, where z_i are samples from the first step of Gibbs sampling iterations.

4. The mean-field variational family of the original model is given by $q(\beta, z) = q(\beta) \prod_{i=1}^N q(z_i)$, where the optimal formula for the two variational distributions are given by:

$$\begin{aligned}
\log q^*(Z) &= \mathbb{E}_{q(\beta)} [\log p(Y, Z, \beta \mid x)] \\
&= \mathbb{E}_{q(\beta)} \left[-\frac{1}{2} \beta^\top \Sigma_b^{-1} \beta + \sum_{i=1}^N y_i \log \mathbb{1}(z_i > 0) + (1 - y_i) \log \mathbb{1}(z_i \leq 0) - \frac{1}{2} (z_i - x_i \beta)^2 + C_2 \right] \\
&= \sum_{i=1}^N y_i \log \mathbb{1}(z_i > 0) + (1 - y_i) \log \mathbb{1}(z_i \leq 0) - \frac{1}{2} (z_i^2 - 2x_i \mu_\beta z_i) + C_6,
\end{aligned}$$

where, $\mu_\beta = \mathbb{E}_{q(\beta)}[\beta]$.

Therefore, the variational distribution of Z is given by

$$q(Z) = \prod_{i=1}^N \mathbb{1}(z_i > 0)^{y_i} \mathbb{1}(z_i \leq 0)^{1-y_i} \mathcal{N}(x_i \mu_\beta, 1). \quad (3)$$

Similarly, the optimal formula for the distribution of β is given by:

$$\begin{aligned}\log q^*(\beta) &= \mathbb{E}_{q(Z)}[\log p(Y, Z, \beta|x)] \\ &= \mathbb{E}_{q(Z)} \left[-\frac{1}{2} \beta^T \Sigma_b^{-1} \beta + \sum_{i=1}^N y_i \log \mathbb{1}(z_i > 0) + (1 - y_i) \log \mathbb{1}(z_i \leq 0) - \frac{1}{2} (z_i - x_i \beta)^2 + C_2 \right] \\ &= -\frac{1}{2} \beta^T \Sigma_b^{-1} \beta - \frac{1}{2} \sum_{i=1}^N (\beta^T x_i^T x_i \beta - 2 \mu_{z_i} x_i \beta) + C_7\end{aligned}$$

Therefore, the variational distribution of β is given by

$$q(\beta) = \mathcal{N} \left(\Sigma_q \left(\sum_{i=1}^N \mu_{z_i} x_i^T \right), \Sigma_q \right), \quad \text{where } \Sigma_q = \left(\sum_{i=1}^N x_i^T x_i + \Sigma_b \right)^{-1} \quad (4)$$

To implement the variational inference algorithm, we can leverage the fact that the density in Equation (3) is a truncated normal distribution, given by

$$q(z_i) = \begin{cases} \mathcal{N}_{[0, \infty]}(x_i \mu_\beta, 1) & \text{if } y_i = 1, \\ \mathcal{N}_{[-\infty, 0]}(x_i \mu_\beta, 1) & \text{otherwise.} \end{cases}$$

Therefore, the mean and variance of the truncated normal distributions can be calculated by

$y_i = 1 :$

$$\mu_{z_i} = x_i \mu_\beta + \frac{\phi(-x_i \mu_\beta) - 0}{1 - \Phi(-x_i \mu_\beta)} \sigma = x_i \mu_\beta + \frac{\phi(-x_i \mu_\beta)}{1 - \Phi(-x_i \mu_\beta)},$$

$$\text{Var}(z_i) = 1 + x_i \mu_\beta \frac{\phi(-x_i \mu_\beta)}{1 - \Phi(-x_i \mu_\beta)} - \left(\frac{\phi(-x_i \mu_\beta)}{1 - \Phi(-x_i \mu_\beta)} \right)^2$$

$y_i = 0 :$

$$\mu_{z_i} = x_i \mu_\beta + \frac{0 - \phi(-x_i \mu_\beta)}{\Phi(-x_i \mu_\beta)} \sigma = x_i \mu_\beta - \frac{\phi(-x_i \mu_\beta)}{\Phi(-x_i \mu_\beta)},$$

$$\text{Var}(z_i) = 1 - x_i \mu_\beta \frac{\phi(-x_i \mu_\beta)}{\Phi(-x_i \mu_\beta)} - \left(\frac{\phi(-x_i \mu_\beta)}{\Phi(-x_i \mu_\beta)} \right)^2$$

Therefore, the variational inference algorithm can be implemented as follows:

```
vb_probit <- function(x, y, mu_beta, cov_beta, max_iter = 100, atol = 1e-6) {
  # initialization
  mu_z <- rep(0.0, nrow(x))
  var_z <- rep(1.0, nrow(x))
  iter <- 0
  while (TRUE) {
    # Update variational distribution of Z
    phi_pdf <- pmax(dnorm(-x %*% mu_beta), 1e-6)
    phi_cdf <- pmax(pnorm(-x %*% mu_beta), 1e-6)
    new_mu_z <- ifelse(
      y == 1,
      x %*% mu_beta + phi_pdf / (1 - phi_cdf),
      x %*% mu_beta - phi_pdf / phi_cdf
    )
    new_var_z <- ifelse(
      y == 1,
      1 + x %*% mu_beta * phi_pdf / (1 - phi_cdf) - (phi_pdf / (1 - phi_cdf))^2,
```

```

    1 - x %*% mu_beta * phi_pdf / phi_cdf - (phi_pdf / phi_cdf)^2
  )

  # sample from the posterior distribution of beta
  new_cov_beta <- solve(t(x) %*% x + cov_beta)
  new_mu_beta <- new_cov_beta %*% rowSums(t(x) %*% new_mu_z)

  if (
    iter > max_iter ||
    all(abs(new_mu_beta - mu_beta) < atol) ||
    all(abs(new_cov_beta - cov_beta) < atol) ||
    all(abs(new_mu_z - mu_z) < atol) ||
    all(abs(new_var_z - var_z) < atol)
  ) {
    return(
      list(mu_beta = mu_beta, cov_beta = cov_beta, mu_z = mu_z, var_z = var_z)
    )
  }

  mu_beta <- new_mu_beta
  cov_beta <- new_cov_beta
  mu_z <- new_mu_z
  var_z <- new_var_z
  iter <- iter + 1
}
}

```

Effect of Computer-generated Reminders

An experiment was conducted to study the effects of computer-generated reminders on preventive care. Physicians (specifically, residents and faculty members on the staff of a general medicine clinic) were randomly assigned to either a treatment or control group. The treatment in this experiment is a reminder sent to physicians that encouraged them to inoculate their patients at risk for the flu. The question of interest here is **whether the computer-generated reminder increases the likelihood that a patient will be vaccinated**.

Your task is to use a **probit regression model** to learn about the effect of the treatment on patient vaccination, accounting for patient characteristics that may be associated with the response. The observed data is contained in `flu_data.txt`, and consists of the following variables for each of the 2,901 patients. You can drop *patients with missing covariates* from your analysis

- **treatment**: An indicator that equals 1 if the patient's doctor received treatment, and 0 otherwise.
 - **vaccinated**: An indicator that equals 1 if the patient received the vaccine, and 0 otherwise.
 - **age**: The patient's age.
 - **copd**: Equals 1 if the patient has chronic obstructive pulmonary disease, and 0 otherwise.
 - **heartd**: An indicator that equals 1 if the patient has heart disease, and 0 otherwise.
 - **renal**: An indicator that equals 1 if the patient has kidney disease, and 0 otherwise.
 - **liverd**: An indicator that equals 1 if the patient has liver disease, and 0 otherwise.
1. (25 points) Apply your function from the previous question on the data provided in `flu_data.txt`. Describe how you initialized the algorithm, the number of iterations that you ran it for, and your selected convergence criterion. Plot the approximation to the posterior distribution over β as well as a few of the z_i 's. Based on your posterior approximation, what do you conclude about the effect of the treatment on patient vaccination?
 2. (20 points) Implement the probit model in Stan, and produce an MCMC approximation to the posterior. Treating these as the truth (recall MCMC becomes arbitrarily accurate with increasing number of samples), comment on your variational approximation deviates from the true posterior distribution.
 3. (20 points) Implement the *logistic regression* model in Stan, and produce an MCMC approximation to the posterior. Comment on any difference between this as the results from the probit model.

Solution:

We first load the data, where the target variables y_i are the `vaccinated` column, and the covariates x_i are the remaining columns. Then, rows with missing covariates are removed from the loaded dataset. In this case, particularly, we remove any row with its age values “.”.

```
if (file.exists("data/flu_data.txt")) {  
  df <- read.table("data/flu_data.txt", header = TRUE, sep = " ")  
  sprintf("Successfully loaded %d rows of data", nrow(df))  
} else {  
  stop("FileNotFound: data file not found at 'data/flu_data.txt'.")  
}
```

```
## [1] "Successfully loaded 2901 rows of data"
```

```
df <- df[!df$age == ".", ]  
df$age <- as.integer(as.character(df$age))  
sprintf("Keep %d rows of data after removing missing covariates", nrow(df))
```

```
## [1] "Keep 2893 rows of data after removing missing covariates"
```

For applying the variational inference algorithm, the mean and covariance of β are initialized with random values and identity matrix. The algorithm is implemented as follows:

```
# initialization  
x <- as.matrix(df[, -c(2)])
```

```

y <- df$vaccinated
mu_beta <- rnorm(ncol(x), 0, 0.01)
cov_beta <- diag(nrow = ncol(x), ncol = ncol(x))
result <- vb_probit(x = x, y = y, mu_beta = mu_beta, cov_beta = cov_beta)

```

Next, we implement the probit model in Stan following the official instruction¹. The code is as shown below:

```

probit_code <- "
  data {
    int<lower=0> n;                // number of observations
    int<lower=0> k;                // number of predictors/features
    real<lower=0> pr_std;          // prior standard deviation for beta
    matrix[n, k] x;              // predictor matrix
    array[n] int<lower=0,upper=1> y; // binary outcome variable
  }

  parameters {
    vector[k] beta;              // regression coefficients
  }

  transformed parameters {
    vector[n] mu;                // linear predictor
    mu = x * beta;
  }

  model {
    beta ~ normal(0, pr_std);
    y ~ bernoulli(Phi(mu));
  }
"
probit_model <- stan_model(
  model_name = "probit_regression",
  model_code = probit_code
)
x <- as.matrix(df[, -c(2)])
y <- df$vaccinated
reg_data <- list(n = nrow(x), k = ncol(x), pr_std = 0.01, x = x, y = y)
nfit_probit <- sampling(
  probit_model,
  data = reg_data,
  chains = 1,
  iter = 5000,
  warmup = 2000,
  seed = 42,
  show_messages = FALSE
)

```

Finally, we implement the logistic regression model following the official instruction². The code is as shown below:

```

logistic_code <- "
  data {
    int<lower=0> n;                // number of observations

```

¹https://mc-stan.org/docs/2_21/stan-users-guide/logistic-probit-regression-section.html

²https://mc-stan.org/docs/2_21/stan-users-guide/logistic-probit-regression-section.html

```

    int<lower=0> k;                // number of predictors/features
    real<lower=0> pr_std;          // prior standard deviation for beta
    matrix[n, k] x;              // predictor matrix
    array[n] int<lower=0,upper=1> y; // binary outcome variable
}

parameters {
    vector[k] beta;              // regression coefficients
}

transformed parameters {
    vector[n] mu;                // linear predictor
    mu = x * beta;
}

model {
    beta ~ normal(0, pr_std);
    y ~ bernoulli_logit(mu);
}
"
logistic_model <- stan_model(
  model_name = "logistic_regression",
  model_code = logistic_code
)
nfit_logistic <- sampling(
  logistic_model,
  data = reg_data,
  chains = 1,
  iter = 5000,
  warmup = 2000,
  seed = 42,
  show_messages = FALSE
)

```

For comparison, we visualize the posterior distribution of β estimated by the variational inference algorithm and the MCMC approximation from the probit and logistic regression models. The results are shown below:

```

# plot the posterior distribution of beta
probit_samples <- as.data.frame(nfit_probit)[, seq_len(ncol(x))]
probit_samples$label <- "Probit"
logistic_samples <- as.data.frame(nfit_logistic)[, seq_len(ncol(x))]
logistic_samples$label <- "Logistic"
combined_samples <- rbind(probit_samples, logistic_samples)
colnames(combined_samples)[seq_len(ncol(x))] <- colnames(x)

plot_list <- list()
for (i in seq_len(ncol(x))) {
  dim_name <- colnames(combined_samples)[i]
  x_range = c(
    result$mu_beta[i] - sqrt(result$cov_beta[i, i]),
    result$mu_beta[i] + sqrt(result$cov_beta[i, i])
  )
  p <- ggplot(combined_samples, aes_string(x = dim_name, fill = "label")) +
    stat_function(

```

```

    fun = dnorm,
    aes(group = NULL),
    args = list(mean = result$mu_beta[i], sd = sqrt(result$cov_beta[i, i])),
    color = "red",
    inherit.aes = FALSE
  ) +
  geom_density(alpha = 0.5, bins=30, position="identity") +
  labs(
    title = dim_name,
    x = dim_name,
    y = "Density"
  ) +
  scale_fill_manual(values = c("blue", "green")) +
  # scale_x_continuous(limits = x_range)
  theme_minimal() +
  theme(legend.position = "top")
plot_list[[i]] <- p
}

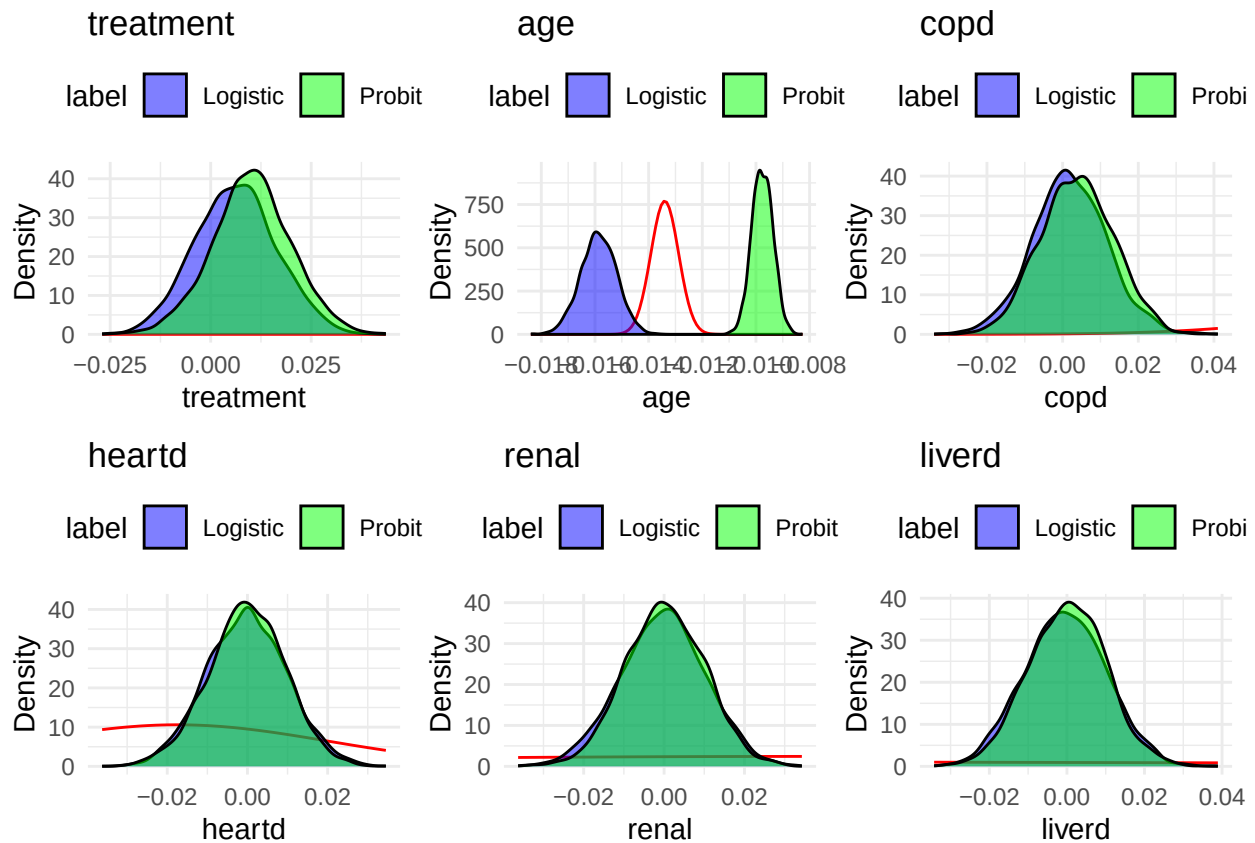
```

```

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

## Warning in geom_density(alpha = 0.5, bins = 30, position = "identity"): Ignoring unknown parameters:
## Ignoring unknown parameters: `bins`
## Ignoring unknown parameters: `bins`
## Ignoring unknown parameters: `bins`
## Ignoring unknown parameters: `bins`
## Ignoring unknown parameters: `bins`
grid.arrange(grobs = plot_list, ncol = 3)

```



Based on the results:

1. The treatment overall has a **positive effect** on patient vaccination, as suggested by the positive mean of the variational approximation of β_1 and the fact that 0.0 is not included in the 95% credible interval of the variational distribution $q(\beta_1)$.
2. The variational approximation deviates from the true posterior distribution given by MCMC, as means and covariance matrices are significantly different from each other indicated by the visualization and the table below.
3. The logistic regression model provides similar results to the probit model, except for the posterior distribution of age, where the logistic model has a lower coefficient than the probit model.

```
probit_mean <- colMeans(probit_samples[, seq_len(ncol(x))])
probit_var <- sapply(probit_samples[, seq_len(ncol(x))], var)
logistic_mean <- colMeans(logistic_samples[, seq_len(ncol(x))])
logistic_var <- sapply(logistic_samples[, seq_len(ncol(x))], var)
data.frame(
  Variable = colnames(x),
  VB_Mean = result$mu_beta,
  VB_Var = diag(result$cov_beta),
  Probit_Mean = probit_mean,
  Probit_Var = probit_var,
  Logistic_Mean = logistic_mean,
  Logistic_Var = logistic_var
)
```

##	Variable	VB_Mean	VB_Var	Probit_Mean	Probit_Var
##	treatment	0.29631701	1.344320e-03	0.0101924971	9.654961e-05
##	age	-0.01339860	2.682493e-07	-0.0097584108	1.563706e-07

```

## copd          copd  0.11916733 1.621378e-03  0.0038187767 9.371704e-05
## heartd        heartd -0.01752265 1.417141e-03 -0.0001262495 9.095832e-05
## renal         renal  0.04174624 2.687137e-02  0.0001915104 9.696222e-05
## liverd        liverd -0.24112575 1.100647e-01  0.0001180196 9.916200e-05
##              Logistic_Mean Logistic_Var
## treatment    0.0062093161 1.016115e-04
## age          -0.0158654691 4.512002e-07
## copd          0.0018900892 9.718530e-05
## heartd        -0.0002179763 1.001584e-04
## renal         -0.0002810725 1.103661e-04
## liverd        -0.0003725746 1.096035e-04

```